

RAPPORT DE PROJET

Gestionnaire de QCM en langage C

Alaa OUAZBIR & Aryem ZITOUNE

Projet de programmation M14 — Langage C
2025 – 2026

1. Présentation de l'équipe

Ce projet a été réalisé en binôme . Chaque membre a pris en charge une partie précise du développement, tout en réfléchissant ensemble à la conception globale du programme.

Membre	Responsabilités
Alaa OUAZBIR	Partie étudiant, mise en œuvre de la fonction QCM (passage, notation, chargement)
Aryem ZITOUNE	Partie enseignant, fonction main, gestion du mot de passe et sauvegarde des QCM

2. Description du projet

L'objectif de ce projet est de développer un gestionnaire de QCM (Questionnaire à Choix Multiples) en langage C. L'application propose deux modes de fonctionnement distincts :

- Mode enseignant (protégé par mot de passe) : création et sauvegarde de QCM paramétrables.
- Mode étudiant : passage d'un QCM existant avec notation automatique.

Les QCM proposent plusieurs paramètres configurables :

- Points négatifs : activation ou non d'une pénalité pour les mauvaises réponses.
- Multi-réponses : possibilité de sélectionner plusieurs réponses par question.
- Mode séquentiel : obligation de répondre à chaque question avant de passer à la suivante.

- Nous avons également ajouté l'affichage immédiat de "Bonne réponse" ou "Mauvaise réponse" après chaque question, afin que l'étudiant puisse identifier les questions auxquelles il a mal répondu.

Les QCM sont sauvegardés dans des fichiers texte dans un dossier sauvegarde/, ce qui permet de les conserver entre les sessions.

3. Organisation et flux de travail

Avant de débiter le codage, nous avons réfléchi ensemble à l'organisation générale du projet. Nous avons rapidement réalisé que la gestion des QCM et le mode enseignant étaient prioritaires, car ils conditionnaient la création des fichiers nécessaires au développement de la partie étudiant.

Par la suite, chaque membre de l'équipe a travaillé sur son code de son côté, sur son propre PC. N'étant pas pleinement à l'aise avec les fonctionnalités de GitHub au début du projet, l'essentiel de notre travail s'est fait sur nos ordinateurs (ce qui explique le faible nombre de commits). Nous nous synchronisons très régulièrement via WhatsApp pour s'entraider dès qu'on était coincés et vérifier que nos morceaux de code allaient bien s'assembler. Une fois que tout marchait bien et que chacun avait validé son travail de son côté, nous avons centralisé l'ensemble du code sur GitHub pour finaliser le rendu.

4. Problèmes rencontrés et solutions apportées

Le développement a présenté de nombreuses problématiques techniques. Voici les principales, accompagnées des solutions mises en place :

Problème 1 - Saisie utilisateur non sécurisée

Problème : L'utilisation de `scanf` seul s'avérait dangereuse : si l'utilisateur entrait une valeur inattendue (lettre au lieu d'un chiffre), le programme pouvait boucler indéfiniment ou planter.

Solution : Création d'une fonction `scanf_secu` combinant `fgets`, `sscanf` et une boucle de validation. Cette fonction lit une ligne entière, tente de parser un entier, vérifie les bornes, et recommence si la saisie est invalide.

Problème 2 - Débordement de buffer (buffer overflow)

Problème : Lorsqu'un utilisateur entrait plus de caractères que prévu, le reste restait dans le buffer `stdin` et était relu automatiquement aux appels suivants, provoquant des comportements imprévisibles.

Solution : Après chaque `fgets`, on vérifie si le dernier caractère est `'\n'`. Si ce n'est pas le cas, on vide le reste du buffer avec une boucle `getchar()`.

Problème 3 - Incompatibilité Windows / Linux

Problème : La fonction `system("ls sauvegarde/ > liste.txt")` ne fonctionnait pas sous Windows. De même, la création de dossier avec `mkdir` prend deux arguments sous Linux mais un seul sous Windows.

Solution : Utilisation de la bibliothèque dirent.h pour parcourir les dossiers de manière portable. Des directives #ifdef _WIN32 permettent de choisir automatiquement la bonne version de mkdir selon le système.

Problème 4 - Dossier sauvegarde inexistant au premier lancement

Problème : Au premier démarrage, le dossier sauvegarde/ n'existait pas, ce qui faisait échouer silencieusement tous les appels à fopen.

Solution : Création d'une macro CREER_DOSSIER dans qcm.h, appelée dans le main, qui crée automatiquement le dossier si nécessaire, de façon compatible Windows et Linux.

Problème 5 - Symétrie lecture / écriture des fichiers QCM

Problème : Si l'ordre d'écriture et de lecture dans les fichiers .qcm ne correspondaient pas exactement, le chargement produisait des données corrompues de façon silencieuse.

Solution : Définition du format ligne par ligne avant de coder, puis test de la symétrie avec un QCM codé en dur. Toutes les lectures se font uniquement avec fgets + sscanf.

5. Résultats

À l'issue du développement, le programme est fonctionnel et stable. Voici un bilan des fonctionnalités :

- Mode enseignant opérationnel : création de QCM, sauvegarde dans des fichiers, protection par mot de passe modifiable.
- Mode étudiant opérationnel : affichage de la liste des QCM disponibles, passage du QCM, calcul et affichage de la note sur 20.
- Les 3 paramètres sont gérés : points négatifs, multi-réponses, mode séquentiel.
- Le programme ne plante pas en cas de saisie invalide grâce à scanf_secu.
- Compatible Windows et Linux sans modification du code.

Le projet répond aux exigences principales du cahier des charges. Les deux membres ont assuré leur part du travail et n'ont pas hésité à s'entraider lorsque des problèmes se sont présentés.

